

StatRotationLoadedTime [SimTalk] - Turnplate

Returns the total time during which the *Turnplate* designated by <Path> was rotating while a *MU* was located on the plate.

Syntax

```
<Path>.StatRotationLoadedTime → time
```

Return Value

The return value has the data type *time*.

Example

```
print MyTurnplate.StatRotationLoadedTime
```

See also

Statistics report, [Rotation Time](#)

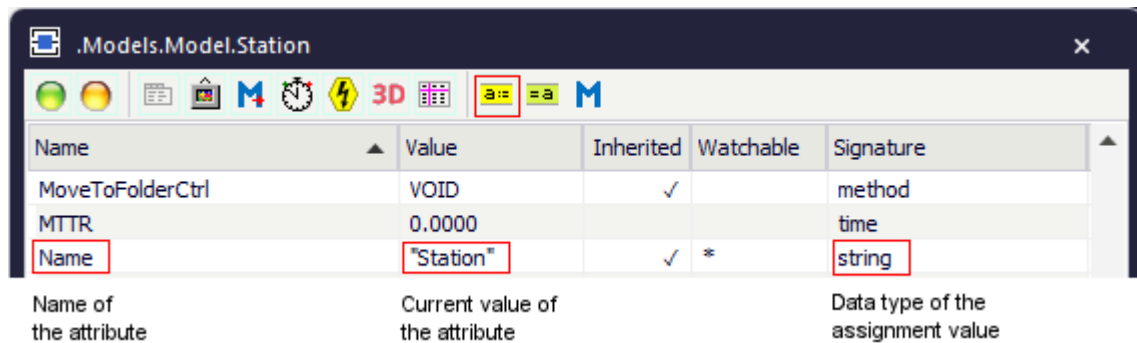
Attributes of the Turnplate

_Attributes of the Turnplate

The *Turnplate* provides:

- The *attributes* listed in the table of contents to the left.
- The [Attributes of All Objects](#).
- The [Attributes of the Material Flow Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyTurnplate.angle := 180
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyTurnplate.angle
posit := MyStation.Cont.XPos
```

Angle [SimTalk] - Turnplate

Sets the rotation angle in degrees to which the *Turnplate* designated by <Path> rotates the *MU*.

Remarks

The rotation angle should be a multiple of 90 degrees. If you specify an angle other than that, *Plant Simulation* rounds this angle to the next angle that is divisible by 90.

You can also specify a value greater than 360 degrees as long as it is divisible by 90. This way the *Turnplate* can rotate the *MU* several times to simulate packing machines.

By default the *Turnplate* rotates 90 degrees clockwise. To rotate counter-clockwise, specify negative angles.

Type

Attribute

Syntax

```
<Path>.Angle:integer
```

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyTurnplate.Angle := 90
```

See also

[Angle \[text box\] - Turnplate](#)

AttributeType [SimTalk] - Turnplate

Sets the **Attribute Type** of the *Turnplate* designated by <Path>.

Remarks

AttributeType applies to the **Strategy > MU Attribute**.

Type

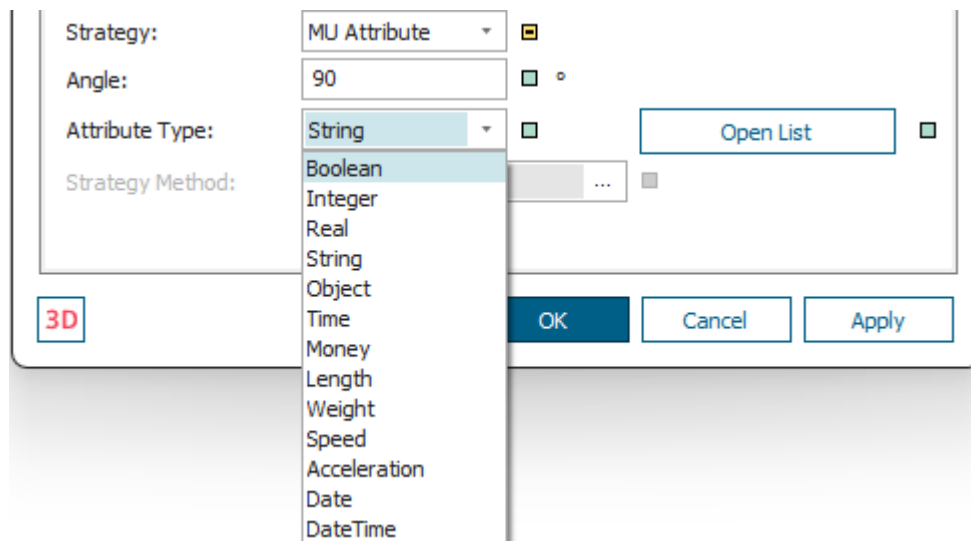
Attribute

Syntax

```
<Path>.AttributeType:string
```

Assignment Value

You can assign a value of data type *string*.



Example

```
MyTurnplate.AttributeType := "String"
```

See also

[Attribute Type \[drop-down list\] - Turnplate](#)

[Strategy \[drop-down list\] - Turnplate](#)

AutomaticStop [SimTalk] - Turnplate

Automatically stops the *Turnplate* designated by<Path>. This means that it sets the current speed to 0 when it does not transport a *MU*.

Remarks

This might, for example, be the case if it is empty or if it is blocked when a *MU* cannot leave it.

If the speed of the *Turnplate* is 0, its **energy state** changes to **operational**.

Type

Attribute

Syntax

```
<Path>.AutomaticStop:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyTurnplate.AutomaticStop := true
```

See also

[Automatic Stop \[check box\] - Turnplate](#)

[Operational \[energy\]](#)

Length [Turnplate]

Sets the **Length** of the *Turnplate* designated by <Path>.

Remarks

The *Turnplate* only rotates *MUs*, which it can accommodate in their entirety, meaning that they are shorter or as long as the value you enter here.

Type

Attribute

Syntax

```
<Path>.Length: length
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *length*.

Note

In *SimTalk* 2.0 you can specify the length units m, mm, km, cm, yd, ft, and in.

Type in the unit directly after the values, without a separating blank space, for example 10m or 10.2m.

You can specify the unit for *floating point values* and for *integer values*.

Example

```
MyTurnplate.Length := 2.0m
```

See also

[Length \[text box\] - Turnplate](#)

RotationTimePer90Degrees [SimTalk] - Turnplate

Sets the time it takes the *Turnplate* designated by <Path> to rotate by a rotation step of 90 degrees.

Remarks

Specify a rotation time of 0 to rotate the *MU* immediately, without using up any time at all.

Type

Attribute

Syntax

```
<Path>.RotationTimePer90Degrees:time
```

Assignment Value

You can assign a value of data type *time*.

Example

```
MyTurnplate.RotationTimePer90Degrees := 0.75
```

See also

[Rotation Time per 90° \[text box\] - Turnplate](#)

Speed [SimTalk] - Turnplate

Sets the speed, with which the *Turnplate* designated by <Path> transports the *MU*, while it is located on the plate.

Remarks

Specify -1 for an infinite speed.

Type

Attribute

Syntax

```
<Path>.Speed: speed
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *speed*.

Note

In *SimTalk* 2.0 you can specify the speed units mps, fps, kmh, and mph.

Type in the unit directly after the number, without a separating blank space, for example 100kmh or 100.5kmh.

```
var len := 1.0ft  
var s : speed := 10.5m / 1:30  
var x : length := 3m
```

Note

In *SimTalk* 2.0 you can enter the unit s for **seconds**. Enter the unit s directly after the number, without a separating blank space. Entering the unit sometimes is necessary to make sure that the computed expression has the correct data type.

```
var x : length := 10m
var s : speed := x / 2s    // Beware: x/2 would have the wrong unit, m
                           instead of m/s
```

Example

```
MyTurnplate.Speed := 1.5
```

See also

[Speed \[text box\] - Turnplate](#)

Strategy [SimTalk] - Turnplate

Sets the strategy according to which the *Turnplate* designated by <Path> rotates the *MU*.

Type

Attribute

Syntax

```
<Path>.Strategy:string
```

Assignment Value

You can assign a value of data type *string*.

You can specify:

- "Angle" rotates the *MU* according to the rotation *Angle* you specify.
- "MU Attribute" rotates the *MU* according to a built-in or a user-defined attribute of the part.
- "MU Name" rotates the *MU* according to its name.
- "Method" rotates the *MU* according to the *strategy method*.

Within this method you have to call the method *rotatePart* with the rotation angle as parameter.

Example

```
MyTurnplate.Strategy := "MU Attribute"
```

SimTalk

[Angle \[SimTalk\] - Turnplate](#)

[rotatePart \[SimTalk\]](#)

See also

[Strategy \[drop-down list\] - Turnplate](#), drop-down list

[Angle \[text box\] - Turnplate](#)

StrategyCtrl [SimTalk] - Turnplate

Designates a *Method* object of the object designated by <Path>. Type in the source code that determines the rotation angle around which the *Turnplate* designated by <Path> rotates the *MU*.

Remarks

The *Turnplate* calls the *Method* for the **Strategy Method** as soon as the booking point of the *MU* is located on the center of rotation of the *Turnplate*. Within this method you have to call the method *rotatePart* with the rotation angle as parameter.

The **default strategy method** looks like this:

```
?.rotatePart(90)
```

Type

Attribute

Syntax

```
<Path>.StrategyCtrl:method
```

Assignment Value

You can assign a value of data type *method*.

Examples

```
var rotAngle: integer
if @.typeOf(~.MyPart)           // rotates the part
    rotAngle := 90
else
    if @.typeOf(~.MyPallet)      // rotates the pallet
        rotAngle := -(4 * 360)  // minus (-) designates counterclockwise
    end
end
?.rotatePart(rotAngle)
```

```
MyTurnplate.StrategyCtrl := &myRotationStrategy
```

SimTalk

[rotatePart \[SimTalk\]](#)

See also

[Strategy Method \[Turnplate\]](#)

Width [SimTalk] - Turnplate

Sets the **Width** of the *Turnplate* designated by <Path>.

Type

Attribute

Syntax

```
<Path>.Width:length
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *length*.

Note

In *SimTalk* 2.0 you can specify the length units m, mm, km, cm, yd, ft, and in.

Type in the unit directly after the values, without a separating blank space, for example 10m or 10.2m.

You can specify the unit for *floating point values* and for *integer values*.

Example

```
MyTurnplate.Width := 2 // meters
```

See also

[Width \[text box\] - Turnplate](#)

Track

Track [object]

Use the object *Track* for modeling way to model a part of a transport line, with or without **automatic routing**, on which the *Transporter* drives and moves parts.

Image



Description

You might, for example, use the *Track* and the *Transporter* to model an AGV (automated guided vehicle) system. The distance, which the **Transporter** 🚚 has to travel on the *Track*—defined by the *Track*'s **Length**, the *Transporter*'s **MU Length**, and its **Speed**—determines the time, which the *Transporter* remains on the *Track*. As opposed to the point-oriented material flow objects, *Plant Simulation* uses the actual **Length**, which you type in, during your simulation run. A *Transporter* cannot pass another *Transporter* moving in front of it. *Transporters* thus retain their order of moving onto and of moving off the *Track* (FIFO)

If several *Transporters* travel along the *Track* at a different speed, and a faster one collides with a slower one, *Plant Simulation* activates the **Collision Control** of the faster *Transporter* and automatically reduces its speed to the speed of the slower *Transporter*.

The maximum capacity of the *Track* is defined by its length and the lengths of the individual *Transporters* moving on it, i.e., a *Track* that is three yards long accepts three *Transporters* of one yard each at the most. You can use the **Capacity** to further restrict the number of *Transporters* located on the *Track*.

The *Transporter* can drive forward and **Backwards** on the *Track*, i.e., it drives onto the *Track* at its **Exit** and exits it at its **Entrance**. Driving forward and backward are not properties of the *Track*, but of the *Transporters*, which drive on it.

You can insert the *Track* into a *Frame* as follows:

- As a curved object, which is the default setting.
- By inserting any sequence of curved segments and straight segments, you can realistically model curved conveyor systems on which the *Transporters* move.

You can select different configurations for the *Track* on the tab **Appearance**.